

RBE 500 – Homework 3

Tracy Yang

June 22, 2026

Part 1: Forward Kinematics / DH Parameters

Task 1: Coordinate Frame Assignment (D-H Convention)

Frames are assigned to the ABB IRB1400 as follows. All six joints are revolute.

- **Frame 0 (base):** Fixed world frame. The z_0 axis points vertically upward along the axis of Joint 1. The origin o_0 is at the base of the robot on the ground.
- **Frame 1:** Attached to the output of Joint 1. z_1 is aligned with the axis of Joint 2 (horizontal, perpendicular to the plane of the arm). The origin o_1 is 475 mm above and 150 mm laterally offset from o_0 .
- **Frame 2:** Attached to the output of Joint 2. z_2 is parallel to z_1 . The link length $a_2 = 600$ mm connects o_1 to o_2 along the upper arm.
- **Frame 3:** Attached to the output of Joint 3. z_3 is perpendicular to z_2 , aligned with the axis of Joint 4. The link length $a_3 = 120$ mm connects o_2 to o_3 .
- **Frame 4:** Located at the center of the spherical wrist. z_4 is aligned with the axis of Joint 5. The offset $d_4 = 720$ mm is the forearm length.
- **Frame 5:** Attached to Joint 5. z_5 is aligned with the axis of Joint 6, perpendicular to z_4 .
- **Frame 6 (end-effector):** Located at the tip of the robot. $d_6 = 170 + 85 = 255$ mm accounts for the wrist-to-tip distance (170 mm wrist body + 85 mm end-effector link).

Assumptions:

- z_0 points vertically upward.
- The 150 mm dimension is taken as the link offset a_1 (perpendicular distance between z_0 and z_1).
- The home position corresponds to all joint variables $\theta_i = 0$.
- The total end-effector offset $d_6 = 170 + 85 = 255$ mm combines the wrist body length and the end-effector link.

Task 2: D-H Parameter Table and Individual Transforms

The standard D-H transformation from frame $i - 1$ to frame i is:

$$T_i^{i-1} = \begin{bmatrix} c\theta_i & -s\theta_i c\alpha_i & s\theta_i s\alpha_i & a_i c\theta_i \\ s\theta_i & c\theta_i c\alpha_i & -c\theta_i s\alpha_i & a_i s\theta_i \\ 0 & s\alpha_i & c\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Table 1: D-H Parameters for the ABB IRB1400 (all distances in mm)

Joint i	θ_i	d_i (mm)	a_i (mm)	α_i
1	q_1	475	150	$+90^\circ$
2	q_2	0	600	0°
3	q_3	0	120	$+90^\circ$
4	q_4	720	0	-90°
5	q_5	0	0	$+90^\circ$
6	q_6	255	0	0°

Individual transformation matrices:

$$T_1^0 = \begin{bmatrix} c_1 & 0 & s_1 & 150 c_1 \\ s_1 & 0 & -c_1 & 150 s_1 \\ 0 & 1 & 0 & 475 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad T_2^1 = \begin{bmatrix} c_2 & -s_2 & 0 & 600 c_2 \\ s_2 & c_2 & 0 & 600 s_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_3^2 = \begin{bmatrix} c_3 & 0 & s_3 & 120 c_3 \\ s_3 & 0 & -c_3 & 120 s_3 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad T_4^3 = \begin{bmatrix} c_4 & 0 & -s_4 & 0 \\ s_4 & 0 & c_4 & 0 \\ 0 & -1 & 0 & 720 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_5^4 = \begin{bmatrix} c_5 & 0 & s_5 & 0 \\ s_5 & 0 & -c_5 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad T_6^5 = \begin{bmatrix} c_6 & -s_6 & 0 & 0 \\ s_6 & c_6 & 0 & 0 \\ 0 & 0 & 1 & 255 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Task 3: Composite Forward Kinematics

The composite transformation from the base frame to the end-effector is:

$$H = T_6^0 = T_1^0 \cdot T_2^1 \cdot T_3^2 \cdot T_4^3 \cdot T_5^4 \cdot T_6^5$$

This was computed numerically in MATLAB using the function below. The general symbolic form is a 4×4 matrix whose upper-left 3×3 block encodes the end-effector orientation and whose right-most column encodes the end-effector position in the base frame.

Listing 1: MATLAB function: standard DH matrix

```

1 function T = dh_matrix(theta, d, a, alpha)
2     % Returns the standard DH homogeneous transformation matrix.
3     % Inputs: theta, alpha in degrees; d, a in mm.
4     theta = deg2rad(theta);
5     alpha = deg2rad(alpha);
6     ct = cos(theta); st = sin(theta);
7     ca = cos(alpha); sa = sin(alpha);
8     T = [ct, -st*ca, st*sa, a*ct;
9          st, ct*ca, -ct*sa, a*st;
10         0, sa, ca, d;
11         0, 0, 0, 1];
12 end

```

Listing 2: MATLAB function: IRB1400 forward kinematics

```

1 function [T, frames] = fk_irb1400(q)
2     % Computes FK for the ABB IRB1400 using standard DH convention.
3     % Input: q = [q1, q2, q3, q4, q5, q6] in degrees
4     % Output: T = 4x4 end-effector pose in base frame
5     % frames = cell array of cumulative transforms {T0, T1, ..., T6}
6
7     % DH table columns: [theta_i, d_i, a_i, alpha_i]
8     dh = [q(1), 475, 150, 90; % Joint 1: base rotation
9          q(2), 0, 600, 0; % Joint 2: shoulder
10         q(3), 0, 120, 90; % Joint 3: elbow
11         q(4), 720, 0, -90; % Joint 4: wrist 1
12         q(5), 0, 0, 90; % Joint 5: wrist 2
13         q(6), 255, 0, 0]; % Joint 6: wrist 3 + EE
14
15     T = eye(4);
16     frames = {T};
17     for i = 1:6
18         Ti = dh_matrix(dh(i,1), dh(i,2), dh(i,3), dh(i,4));
19         T = T * Ti;
20         frames{end+1} = T;
21     end
22 end

```

Question 5: FK for Given Joint Configuration

For $[\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \theta_6] = [0^\circ, 75^\circ, 30^\circ, 135^\circ, -45^\circ, 60^\circ]$, the composite transformation $H = T_6^0$ evaluates to:

$$H = \begin{bmatrix} 0.5647 & -0.6121 & 0.5536 & 1110.87 \\ 0.3624 & 0.7866 & 0.5000 & 127.50 \\ -0.7415 & -0.0817 & 0.6660 & 1526.64 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Position forward kinematics (end-effector origin expressed in the base frame, in mm):

$$\mathbf{p} = \begin{bmatrix} 1110.87 \\ 127.50 \\ 1526.64 \end{bmatrix} \text{ mm}$$

Orientation forward kinematics (rotation matrix from end-effector frame to base frame):

$$R_6^0 = \begin{bmatrix} 0.5647 & -0.6121 & 0.5536 \\ 0.3624 & 0.7866 & 0.5000 \\ -0.7415 & -0.0817 & 0.6660 \end{bmatrix}$$

The columns of R_6^0 give the directions of x_6 , y_6 , z_6 expressed in the base frame.

Question 6: MATLAB 3D Visualization

Listing 3: MATLAB robot visualization (draw_irb1400.m)

```

1 function draw_irb1400(q)
2     % Draws ABB IRB1400 in 3D based on joint configuration.
3     % Input: q = [q1,...,q6] in degrees.
4     % At q = [0,0,0,0,0,0] robot is at defined home position.
5
6     % Compute forward kinematics and extract all frame origins
7     [~, frames] = fk_irb1400(q);
8     n = length(frames);
9     positions = zeros(3, n);
10    for i = 1:n
11        positions(:, i) = frames{i}(1:3, 4);
12    end
13
14    figure; clf; hold on; grid on; axis equal;
15    xlabel('X(mm)'); ylabel('Y(mm)'); zlabel('Z(mm)');
16    title(sprintf('ABB IRB1400 q=[%s] deg', num2str(q, '%.0f')));
17    view(45, 20);
18
19    % Draw links as thick solid lines
20    plot3(positions(1,:), positions(2,:), positions(3,:), ...
21          'b-', 'LineWidth', 3);
22
23    % Draw joint origins as solid red circles

```

```

24 plot3(positions(1,:), positions(2,:), positions(3,:), ...
25         'ro', 'MarkerSize', 10, 'MarkerFaceColor', 'r');
26
27 % Label each frame origin
28 labels = {'o_0', 'o_1', 'o_2', 'o_3', 'o_4', 'o_5', 'o_6'};
29 for i = 1:n
30     text(positions(1,i)+20, positions(2,i)+20, positions(3,i)+20, ...
31          labels{i}, 'FontSize', 9, 'FontWeight', 'bold');
32 end
33 end
34
35 % ---- Run both configurations ----
36 figure(1); draw_irb1400([0, 0, 0, 0, 0, 0]); % home
37 figure(2); draw_irb1400([0, 75, 30, 135, -45, 60]); % question 5

```

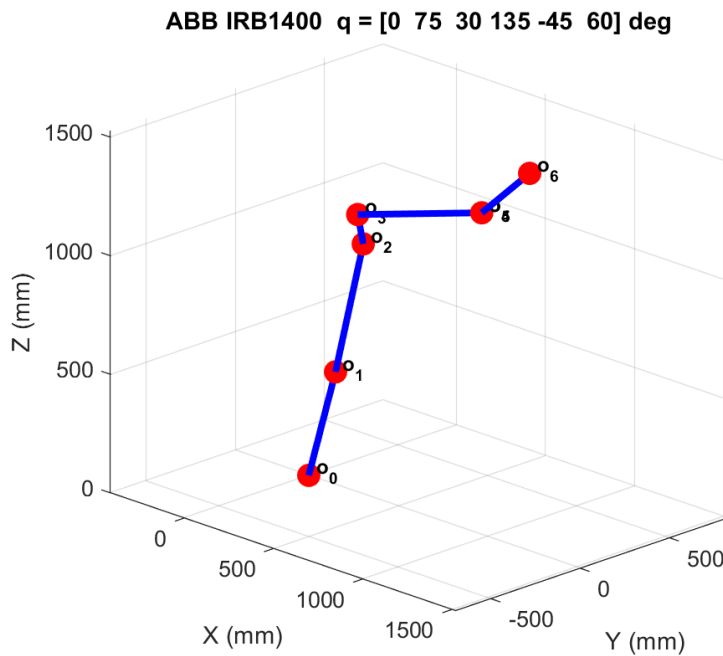


Figure 1: ABB IRB1400 at $[\theta_1, \dots, \theta_6] = [0^\circ, 75^\circ, 30^\circ, 135^\circ, -45^\circ, 60^\circ]$

Question 7 (Extra Credit): Animated Motion to Target Pose

The following MATLAB script animates the robot moving from its home position $[0, 0, 0, 0, 0, 0]^\circ$ to the target configuration $[0, 75, 30, 135, -45, 60]^\circ$ by linearly interpolating the joint angles and redrawing the robot at each intermediate step.

Listing 4: animate_irb1400.m – robot motion animation

```

1 function animate_irb1400(q_start, q_end, n_frames, save_gif)

```

```

2 % Animates the ABB IRB1400 moving from q_start to q_end.
3 % Inputs:
4 % q_start -- initial joint config [q1..q6] in degrees
5 % q_end -- target joint config [q1..q6] in degrees
6 % n_frames -- number of interpolation steps (default: 50)
7 % save_gif -- true/false, whether to save as GIF (default: false)
8
9 if nargin < 3; n_frames = 50; end
10 if nargin < 4; save_gif = false; end
11
12 gif_filename = 'irb1400_animation.gif';
13
14 fig = figure;
15 fig.Color = 'white';
16
17 for k = 1:n_frames
18     alpha = (k - 1) / (n_frames - 1);
19     q = q_start + alpha * (q_end - q_start);
20
21     [~, frames] = fk_irb1400(q);
22     n = length(frames);
23     positions = zeros(3, n);
24     for i = 1:n
25         positions(:, i) = frames{i}(1:3, 4);
26     end
27
28     clf;
29     hold on; grid on; axis equal;
30     xlabel('X(mm)'); ylabel('Y(mm)'); zlabel('Z(mm)');
31     title(sprintf('Frame %d/%d | q=[%s] deg', ...
32         k, n_frames, num2str(round(q), '%d')));
33     view(45, 20);
34     xlim([-500, 1600]); ylim([-800, 800]); zlim([-300, 1800]);
35
36     plot3(positions(1,:), positions(2,:), positions(3,:), ...
37         'b-', 'LineWidth', 3);
38     plot3(positions(1,:), positions(2,:), positions(3,:), ...
39         'ro', 'MarkerSize', 10, 'MarkerFaceColor', 'r');
40
41     T_ee = frames{end};
42     origin = T_ee(1:3, 4);
43     scale = 80;
44     colors = {'r', 'g', 'b'};
45     for ax = 1:3
46         dir = T_ee(1:3, ax) * scale;
47         quiver3(origin(1), origin(2), origin(3), ...
48             dir(1), dir(2), dir(3), 0, ...

```

```

49         'Color', colors{ax}, 'LineWidth', 2, ...
50         'MaxHeadSize', 0.5);
51     end
52
53     labels = {'o_0','o_1','o_2','o_3','o_4','o_5','o_6'};
54     for i = 1:n
55         text(positions(1,i)+20, positions(2,i)+20, ...
56             positions(3,i)+20, labels{i}, ...
57             'FontSize', 8, 'FontWeight', 'bold');
58     end
59
60     drawnow;
61
62     % --- GIF saving ---
63     if save_gif
64         frame = getframe(fig); % capture current figure
65         im = frame2im(frame); % convert to RGB image
66         [imind, cm] = rgb2ind(im, 256); % convert to indexed color
67
68         if k == 1
69             % First frame: create the file
70             imwrite(imind, cm, gif_filename, 'gif', ...
71                 'Loopcount', inf, ... % loop forever
72                 'DelayTime', 0.04); % seconds per frame
73         else
74             % Subsequent frames: append to file
75             imwrite(imind, cm, gif_filename, 'gif', ...
76                 'WriteMode', 'append', ...
77                 'DelayTime', 0.04);
78         end
79     end
80
81     pause(0.04);
82 end
83
84 if save_gif
85     fprintf('GIF saved to: %s\n', gif_filename);
86 end
87 disp('Animation complete. ');
88 end
89
90 % ---- Run and save ----
91 q_home = [0, 0, 0, 0, 0, 0];
92 q_target = [0, 75, 30, 135, -45, 60];
93 animate_irb1400(q_home, q_target, 60, true);

```

Part 2: ROS 2 Node – Forward Kinematics

Overview

This package implements a ROS 2 node (`fk_node`) that subscribes to the `/joint_angles` topic carrying a `Float32MultiArray` message with six values $[q_1, q_2, q_3, q_4, q_5, q_6]$ in degrees. Upon receiving a message, the node computes the forward kinematics of the ABB IRB1400 using the D-H parameters from Part 1, then prints the end-effector position and orientation matrix to the terminal.

Package Structure

```
irb1400_fk/
├── irb1400_fk/
│   ├── __init__.py
│   └── fk_node.py
├── package.xml
├── setup.py
├── setup.cfg
├── resource/
│   └── irb1400_fk
```

Node Implementation

Listing 5: `fk_node.py` – ROS 2 forward kinematics node

```
1 import rclpy
2 from rclpy.node import Node
3 from std_msgs.msg import Float32MultiArray
4 import numpy as np
5
6 class FKNode(Node):
7     def __init__(self):
8         super().__init__('irb1400_fk')
9
10        # Subscribe to /joint_angles (Float32MultiArray: [q1..q6] in degrees)
11        self.subscription = self.create_subscription(
12            Float32MultiArray,
13            'joint_angles',
14            self.listener_callback,
15            10)
16
17        self.get_logger().info('IRB1400_FK_node_started.')
18        self.get_logger().info(
19            'Waiting_for_joint_angles_[q1..q6]_in_degrees...')
```



```

20
21 # ----- #
22 def dh_matrix(self, theta_deg, d, a, alpha_deg):
23     """
24     Compute the standard DH transformation matrix.
25     Parameters: theta, alpha in degrees; d, a in mm.
26     """
27     theta = np.radians(theta_deg)
28     alpha = np.radians(alpha_deg)
29     ct, st = np.cos(theta), np.sin(theta)
30     ca, sa = np.cos(alpha), np.sin(alpha)
31     return np.array([
32         [ct, -st*ca, st*sa, a*ct],
33         [st, ct*ca, -ct*sa, a*st],
34         [0, sa, ca, d],
35         [0, 0, 0, 1]
36     ])
37
38 # ----- #
39 def forward_kinematics(self, q):
40     """
41     Compute FK for the ABB IRB1400 using standard DH convention.
42     Input: q -- list of 6 joint angles in degrees
43     Output: 4x4 homogeneous transformation matrix H = T6^0
44     """
45     # DH table: (theta_i, d_i mm, a_i mm, alpha_i deg)
46     dh_params = [
47         (q[0], 475, 150, 90), # Joint 1: base yaw
48         (q[1], 0, 600, 0), # Joint 2: shoulder pitch
49         (q[2], 0, 120, 90), # Joint 3: elbow pitch
50         (q[3], 720, 0, -90), # Joint 4: wrist roll
51         (q[4], 0, 0, 90), # Joint 5: wrist pitch
52         (q[5], 255, 0, 0), # Joint 6: wrist yaw + EE
53     ]
54     T = np.eye(4)
55     for theta, d, a, alpha in dh_params:
56         T = T @ self.dh_matrix(theta, d, a, alpha)
57     return T
58
59 # ----- #
60 def listener_callback(self, msg):
61     """Receive joint angles, compute FK, and print result."""
62     # Validate message length
63     if len(msg.data) != 6:
64         self.get_logger().error(
65             f'Expected 6 joint angles, got {len(msg.data)}')
66     return

```

```

67     q = list(msg.data)
68
69
70     # Compute forward kinematics
71     H = self.forward_kinematics(q)
72
73     # Extract position vector (mm) and rotation matrix
74     pos = H[:3, 3]
75     R = H[:3, :3]
76
77     # Print end-effector pose to terminal
78     self.get_logger().info(
79         f'\nReceived_joints_(deg):_{[round(qi,2) for qi in q]}\n'
80         f'_{Position_(mm)}:\n'
81         f'_{x=_{pos[0]:.2f}}\n'
82         f'_{y=_{pos[1]:.2f}}\n'
83         f'_{z=_{pos[2]:.2f}}\n'
84         f'_{Rotation_matrix_R_6^0}:\n'
85         f'_{[R[0,0]:7.4f]_{[R[0,1]:7.4f]_{[R[0,2]:7.4f]}\n'
86         f'_{[R[1,0]:7.4f]_{[R[1,1]:7.4f]_{[R[1,2]:7.4f]}\n'
87         f'_{[R[2,0]:7.4f]_{[R[2,1]:7.4f]_{[R[2,2]:7.4f]}}'
88     )
89
90 # ----- #
91 def main(args=None):
92     rclpy.init(args=args)
93     node = FKNode()
94     rclpy.spin(node)
95     node.destroy_node()
96     rclpy.shutdown()
97
98 if __name__ == '__main__':
99     main()

```

How to Run

Build and source the package:

```

cd ~/ros2_ws
colcon build --packages-select irb1400_fk
source install/setup.bash

```

Launch the node:

```

ros2 run irb1400_fk fk_node

```

Publish joint angles from the terminal (six test cases):

```
# Test 1: Home position (all zeros)
ros2 topic pub --once /joint_angles std_msgs/msg/Float32MultiArray \
  "data: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0]"

# Test 2: Question-5 configuration
ros2 topic pub --once /joint_angles std_msgs/msg/Float32MultiArray \
  "data: [0.0, 75.0, 30.0, 135.0, -45.0, 60.0]"

# Test 3: Shoulder 90 deg
ros2 topic pub --once /joint_angles std_msgs/msg/Float32MultiArray \
  "data: [0.0, 90.0, 0.0, 0.0, 0.0, 0.0]"

# Test 4: Base rotated 90 deg
ros2 topic pub --once /joint_angles std_msgs/msg/Float32MultiArray \
  "data: [90.0, 0.0, 0.0, 0.0, 0.0, 0.0]"

# Test 5: Elbow and wrist active
ros2 topic pub --once /joint_angles std_msgs/msg/Float32MultiArray \
  "data: [45.0, 45.0, 45.0, 0.0, 0.0, 0.0]"

# Test 6: All joints active
ros2 topic pub --once /joint_angles std_msgs/msg/Float32MultiArray \
  "data: [30.0, 60.0, -30.0, 90.0, -60.0, 45.0]"
```

Results

```

tracy@motherboard:~/rbe500/ros2_ws$ ros2 run irb1400_fk fk node
[INFO] [1782159514.149944743] [irb1400_fk]: IRB1400 FK node started.
[INFO] [1782159514.150415192] [irb1400_fk]: Waiting for joint angles [q1..q6] in degrees...
[INFO] [1782159524.213787968] [irb1400_fk]:
Received joints (deg): [0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
Position (mm):
x = 870.00
y = -0.00
z = -500.00
Rotation matrix R_6^0:
[ 1.0000  0.0000  0.0000]
[ 0.0000 -1.0000 -0.0000]
[ 0.0000  0.0000 -1.0000]
[INFO] [1782159525.364691380] [irb1400_fk]:
Received joints (deg): [0.0, 90.0, 0.0, 0.0, 0.0, 0.0]
Position (mm):
x = 1125.00
y = -0.00
z = 1195.00
Rotation matrix R_6^0:
[ 0.0000 -0.0000  1.0000]
[ 0.0000 -1.0000 -0.0000]
[ 1.0000  0.0000 -0.0000]
[INFO] [1782159526.535281860] [irb1400_fk]:
Received joints (deg): [90.0, 0.0, 0.0, 0.0, 0.0, 0.0]
Position (mm):
x = 0.00
y = 870.00
z = -500.00
Rotation matrix R_6^0:
[ 0.0000  1.0000  0.0000]
[ 1.0000 -0.0000 -0.0000]
[ 0.0000  0.0000 -1.0000]

```

```

[INFO] [1782159527.695195564] [irb1400_fk]:
Received joints (deg): [45.0, 45.0, 45.0, 0.0, 0.0, 0.0]
Position (mm):
x = 1095.50
y = 1095.50
z = 1019.26
Rotation matrix R_6^0:
[ 0.0000  0.7071  0.7071]
[ 0.0000 -0.7071  0.7071]
[ 1.0000  0.0000 -0.0000]
[INFO] [1782159528.861681919] [irb1400_fk]:
Received joints (deg): [30.0, 60.0, -30.0, 90.0, -60.0, 45.0]
Position (mm):
x = 736.27
y = 680.09
z = 320.66
Rotation matrix R_6^0:
[-0.0884 -0.9723 -0.2165]
[-0.4593 -0.1531  0.8750]
[-0.8839  0.1768 -0.4330]

```

Figure 2: Terminal output showing FK results for all six test cases. *Left*: Tests 1–3 (home, shoulder 90°, base 90°). *Right*: Tests 4–5 (elbow/wrist, all joints active).